

ARCHITECTURE PROPOSAL

AWS Cross-Account Private API Gateway 呼び出し アーキテクチャ提案

Account B Lambda (Cognito Trigger) から Account A Private API を呼ぶ最小構成

01

背景 / 目的

要件・制約・ゴール

02

アーキテクチャ全体図

VPCE 経由のプライベート完結構成

03

Account A がやること

Resource Policy 更新・VPCE-ID 受け入れ

04

Account B がやること

VPCE 作成・Lambda VPC アタッチ・IAM

05

認証方式の比較

AWS_IAM / API Key / なし / Authorizer

06

Lambda Python サンプルコード

SigV4 署名 + VPCE 固有 URL

07

Cognito トリガーとしての注意点

タイムアウト・Cold start・エラー

08

コスト / 監視 / 追加考慮

VPCE 費用・CloudWatch・X-Ray

このページの結論

本資料は背景→構成図→各アカウントの作業→認証→実装→運用の順で読み進められます。

要件

- **Account A:**
Private API Gateway が既に存在
- **Account B:**
Lambda (Python) から A を呼ぶ
- Lambda B は Cognito User Pool の
認証トリガー (例 : Pre Auth)

制約

- **API** はインターネット非公開
- 通信は AWS バックボーン完結
- 同一リージョン・既存 VPC を利用
- Private DNS の cross-account
共有は前提とせず VPCE 固有 URL
を採用する

ゴール

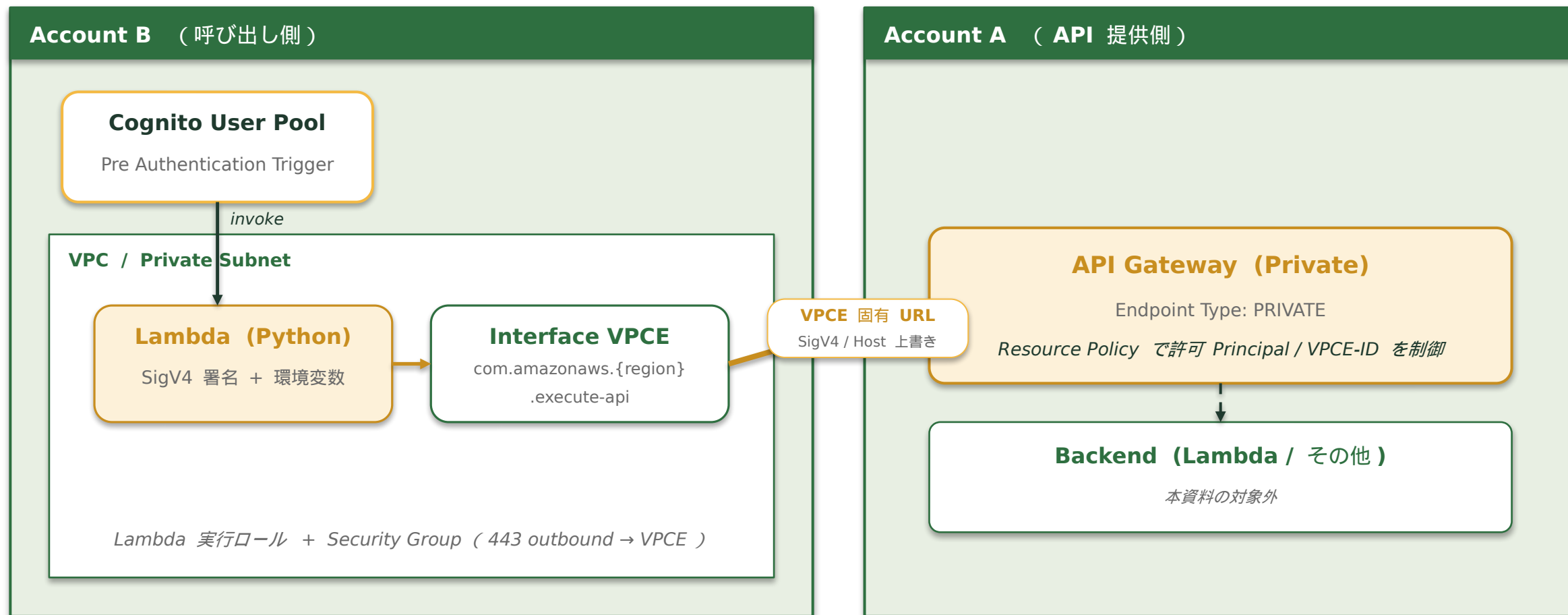
- **B → A** をプライベート経路で
確実に到達できる構成を確立
- 認証 / 認可を明確化し
監査可能な状態にする
- Cognito Trigger の運用制約
(5 秒タイムアウト等) に耐える

このページの結論

Cognito 認証フローの中で、別アカウントの **Private API** を **AWS** バックボーン経由で呼ぶ最小構成を確立する。

アーキテクチャ全体図

4 / 11



— AWS Backbone (インターネット非経由) —

このページの結論

Lambda B → Interface VPC Endpoint → execute-api → Account A の Private API、すべて AWS 内で完結。

Account A がやること

5 / 11

01

Resource Policy の更新

Private API の Resource Policy に Account B の VPCE-ID を許可する。
(Condition: aws:SourceVpce = vpce-xxxxxxx)

02

Principal の追加 (AWS_IAM 採用時)

Action: execute-api:Invoke の Principal に
arn:aws:iam::<account-b>:role/<lambda-role> を許可。

03

VPCE-ID の受け入れ調整

Account B 側で作成された VPCE-ID を共有してもらい、
上記 Resource Policy へ反映する (事前合意)。

04

認証方式に応じた追加対応

API Key を使うなら Usage Plan + Key 発行 / カスタム Authorizer なら Lambda 配備。
認証なしの場合は Resource Policy だけが信頼境界となる点に注意。

このページの結論

Resource Policy で「誰が (**Principal**) どこから (**VPCE-ID**)」呼べるかを制御し、認証方式に応じた追加対応を行う。

Account B がやること

6 / 11

VPC / Subnet

Lambda を配置する VPC を確認。
プライベートサブネット ×2 以上推奨（マルチ AZ）。

Interface VPCE 作成

Service: com.amazonaws.{region}.execute-api
★ Private DNS = OFF（cross-account の鉄則）

Security Group

VPCE 用 SG: 443 inbound from Lambda SG。
Lambda SG: 443 outbound to VPCE SG。

Lambda VPC アタッチ

上記サブネット + Lambda SG に紐付ける。
NAT 不要（VPCE で完結）。

Lambda 実行ロール（IAM）

execute-api:Invoke を該当 API の ARN に許可。
AWS_IAM 採用時のみ必要（他方式では不要）。

環境変数

API_ID / VPCE_ID / STAGE / API_PATH を設定。
リージョンは AWS_REGION（予約済み）を使用。

このページの結論

VPCE 作成（**Private DNS OFF**）と **Lambda** の **VPC** アタッチ。**SigV4** 用 **IAM** ロールと環境変数を整える。

認証方式の比較

7 / 11

方式	概要	セキュリティ	実装難易度	監査容易性
(a) AWS_IAM ★ 推奨	Account A の Resource Policy + Principal に B の Lambda 実行ロール ARN を許可。Lambda 側は SigV4 署名で呼ぶ。	◎ 強い	○ 中程度	◎ CloudTrail
(b) API Key	API Gateway の Usage Plan + API Key。Lambda は x-api-key ヘッダで呼ぶ。Key の Secrets Manager 管理が前提。	△ Key 漏洩リスク	◎ 容易	○ 利用計画ログ
(c) 認証なし	Resource Policy (VPCE-ID + Principal) だけで信頼。VPC 越境がない前提で運用できるが、A の Resource Policy のみが境界となる。	△ 単一防御層	◎ 最小	△ 識別性低
(d) カスタム Lambda Authorizer	Account A 側に Authorizer Lambda を配置し、独自トークンや JWT を検証。柔軟だが運用負荷が増える。	○ 設計依存	△ 高い	○ 自前ログ

判断基準: ① セキュリティ (漏洩耐性・最小権限) ② 実装容易性 (Lambda コードの薄さ) ③ 監査容易性 (誰が呼んだかを後追いできるか)

このページの結論

AWS_IAM を推奨。証跡 (**CloudTrail**) と最小権限制御の両立、追加コンポーネント不要が決め手。

Lambda Python サンプルコード

8 / 11

lambda_function.py - Cognito Pre Authentication Trigger

```
import os, json, boto3, urllib3
from botocore.auth import SigV4Auth
from botocore.awsrequest import AWSRequest

http = urllib3.PoolManager()
API_ID = os.environ["API_ID"]
VPCE_ID = os.environ["VPCE_ID"]
REGION = os.environ["AWS_REGION"]
STAGE = os.environ["STAGE"]
API_PATH = os.environ.get("API_PATH", "/users")

def _signed_request(method, url, body=""):
    creds = boto3.Session().get_credentials().get_frozen_credentials()
    req = AWSRequest(method=method, url=url, data=body)
    SigV4Auth(creds, "execute-api", REGION).add_auth(req)
    return dict(req.headers.items())

def lambda_handler(event, context):
    user_email = event["request"]["userAttributes"].get("email", "unknown")
    url = (f"https://{API_ID}-{VPCE_ID}.execute-api.{REGION}"
          f".amazonaws.com/{STAGE}/{API_PATH}?email={user_email}")
    headers = _signed_request("GET", url)
    # API Gateway は Host で API 識別 → 標準 URL の Host で上書き
    headers["Host"] = f"{API_ID}.execute-api.{REGION}.amazonaws.com"
    resp = http.request("GET", url, headers=headers, timeout=urllib3.Timeout(3.0))
    if resp.status != 200:
        raise Exception(f"upstream {resp.status}: {resp.data[:200]}")
    return event # Cognito は event を返すと認証継続
```

★ 3 点セット：① VPCE 固有 URL ② SigV4 署名 ③ Host ヘッダの上書き

このページの結論

VPCE 固有 URL + SigV4 署名 + Host ヘッダ上書きの 3 点が要点。event をそのまま返せば認証継続。

Cognito トリガーとしての注意点

9 / 11

01

タイムアウト 5 秒の壁

Cognito Trigger は 5 秒以内に Lambda が応答しないと認証失敗。
API 呼び出しは 3 秒タイムアウト (urllib3) で打ち切り、再試行 1 回までに留める。

02

Cold start 対策

VPC アタッチ Lambda は初回起動が遅い。Provisioned Concurrency を最小 1 で常駐させる、
またはランタイムを軽量化 (不要ライブラリを除く) 。

03

エラーハンドリング方針の事前合意

Trigger で例外を投げる = 認証拒否。要件次第で「ログを残して握り潰し→ event 返却」も選択肢。
失敗時の挙動はビジネス判断 (拒否 / 通過) として事前に文書化する。

04

冪等性 / 副作用の整理

Pre Auth は同一セッションで再実行されうる。書き込み系を呼ぶ場合は idempotency key で重複を防ぐ。

このページの結論

5 秒のハードタイムアウトを軸に、**Cold start** とエラー伝播の方針を先に決めてから実装する。

コスト

- Interface VPC: AZ × 時間 + データ処理料金
- 同一 AZ 内呼び出しでデータ転送は最小化
- Lambda: VPC アタッチによる ENI 維持コスト

トレース

- X-Ray 有効化 (Lambda + API GW)
- サブセグメント: SigV4 署名 / HTTP 呼び出し
- 関連 ID は Cognito event['request']['userAttributes'] から付与

監視

- CloudWatch Logs (Lambda) → エラーレート / レイテンシ
- API Gateway 4xx/5xx メトリクス
- VPC Flow Logs (Reject トラフィック検知)

フォールバック / 拡張

- 失敗時挙動の合意 (拒否 / 通過) を IaC コメントで明記
- Account A 側が同一 VPC 内なら VPCE は省略可
- 将来クロスリージョンに広げる場合は Private DNS 戦略を再検討

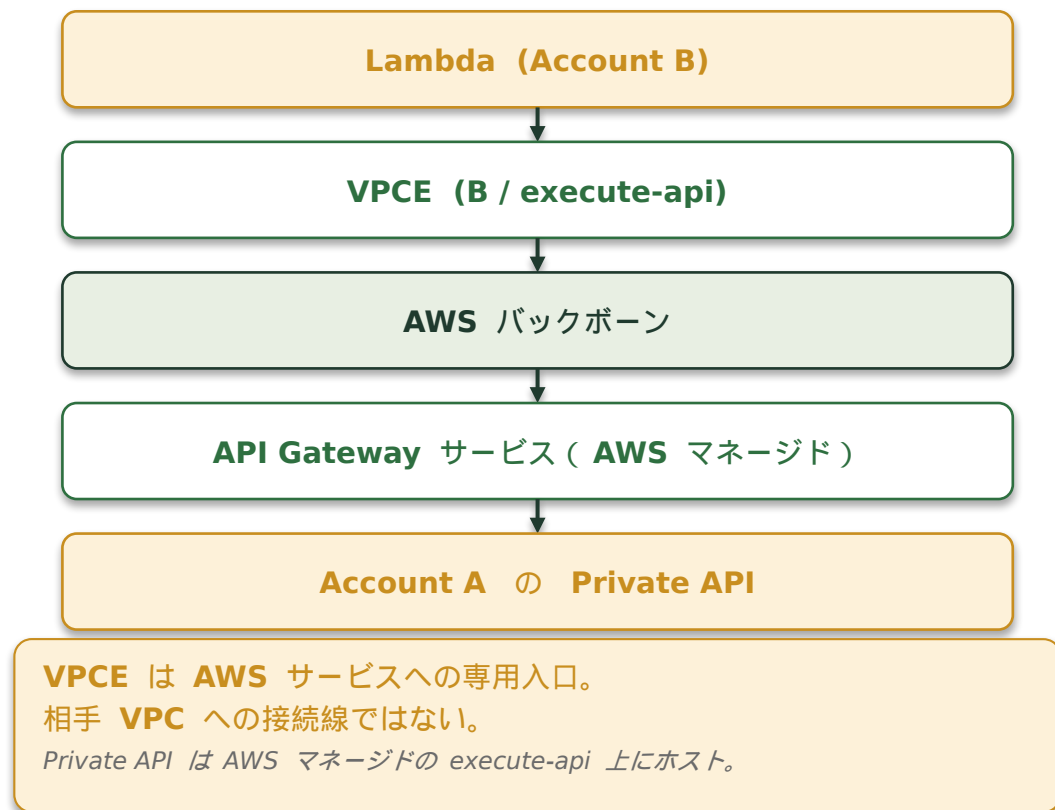
このページの結論

VPCE の常時課金を許容できることが構成成立の前提。監視はメトリクス + 構造化ログ + トレースの三段構え。

Appendix: VPC Peering の要否判断 — 今回は不要、根拠を 3 点で示す

11 / 11

なぜ不要なのか（通信経路の本質）



VPC Peering が必要 vs 不要

ケース	必要？
Private API GW を叩く（今回）	不要 ★
A の VPC 内 RDS / EC2 直接接続	必要
大量サービス・複数アカウント	TGW / PrivateLink 推奨
Private DNS 共有のため	不要（RAM で R53 共有）

Pros（Peering なし）

- 構成シンプル
- CIDR 重複リスク回避
- 最小権限（VPCE-ID で縛れる）

Cons（Peering ありの負荷）

- 両アカウント網設定
- CIDR 重複 NG
- ブラスト半径拡大
- クロス AZ データ転送コスト

このページの結論

Private API GW を叩くだけなら **VPCE** で完結。 **VPC Peering** は将来 **A** の **VPC** 内リソース直接接続が必要になった時に再検討。